1

Software Architecture

Foundations

Includes content adapted from *Fundamentals of Software Architecture – An Engineering Approach*, 2020, O'Reilley

1.2

Architectural Characteristics

Driving Factors for Architecture Design



Architectural Characteristics

- **Architectural Characteristics**
  - Key factors influencing architectural decisions
  - Must be defined, discovered and analysed
  - Related to similar concepts as:
    - Non-functional Requirements
    - Quality Attributes
- **Acs meet these criteria:**
  - Specify a nondomain design consideration
  - Influence structural aspects of the design
  - Are of critical importance for application success



Architectural Characteristics

- Implicit vs. Explicit
  - AC may not always be explicitly discussed in the documentation, but can be derived from it indirectly.
  - Implicit domain knowledge is also important.
- AC groups:
  - Operational ACs
  - Structural ACs
  - Cross-cutting ACs



Operational Architectural Characteristics

Term	Definition
Availability	How long the system will need to be available (if 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure).
Continuity	Disaster recovery capability.
Performance	Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.
Recoverability	Business continuity requirements (e.g., in case of a disaster, how quickly is the system required to be on-line again?). This will affect the backup strategy and requirements for duplicated hardware.
Reliability/ safety	Assess if the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money?
Robustness	Ability to handle error and boundary conditions while running if the internet connection goes down or if there's a power outage or hardware failure.
Scalability	Ability for the system to perform and operate as the number of users or requests increases.



Structural Architectural Characteristics

Term	Definition
Configurability	Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).
Extensibility	How important it is to plug new pieces of functionality in.
Installability	Ease of system installation on all necessary platforms.
Leverageability/ reuse	Ability to leverage common components across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields; on reports, multibyte character requirements and units of measure or currencies.
Maintainability	How easy it is to apply changes and enhance the system?
Portability	Does the system need to run on more than one platform? (For example, does the frontend need to run against Oracle as well as SAP DB?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Upgradeability	Ability to easily/quickly upgrade from a previous version of this application/solution to a newer version on servers and clients.



Cross-cutting Architectural Characteristics

Term	Definition
Accessibility	Access to all your users, including those with disabilities like colorblindness or hearing loss.
Archivability	Will the data need to be archived or deleted after a period of time? (For example, customer accounts are to be deleted after three months or marked as obsolete and archived to a secondary database for future access.)
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, webpage, business rule, field level, etc.).
Legal	What legislative constraints is the system operating in (data protection, Sarbanes Oxley, GDPR, etc.)? What reservation rights does the company require? Any regulations regarding the way the application is to be built or deployed?
Privacy	Ability to hide transactions from internal company employees (encrypted transactions so even DBAs and network architects cannot see them).
Security	Does the data need to be encrypted in the database? Encrypted for network communication between internal systems? What type of authentication needs to be in place for remote user access?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Usability/ achievability	Level of training required for users to achieve their goals with the application/solution. Usability requirements need to be treated as seriously as any other architectural issue.



Trade-offs and Least Worst Architecture

- **Only a few ACs should be supported because:**
  - Addressing an AC requires design effort, structural support and has costs.
  - One AC can negatively impact another. *(e.g security negatively impacts performance)*
- **Every option is a trade-off:**
  - Seek the Least-Worst Architecture, rather than the “best” architecture.

1.2

Architectural Characteristics

Driving Factors for Architecture Design



Identifying Architectural Characteristics

- **ACs are derived from three sources:**
  - **Domain Concerns –** Collaborating with Stakeholders, with the purpose of identifying the most important Acs.
  - **Requirements** – Explicitly stated information in requirements
  - **Domain Knowledge** – Complements and completes other sources of domain information.



Identifying Architectural Characteristics from Domain Concerns

- **Stakeholders are asked to identify the most important aspects of the application and business.**
- **The final list of AC should be as small as possible.**
  - Stakeholders may not reach a consensus on the correct ordering of priorities.
  - Stakeholders can be oblivious to the negative impacts of supporting many AC.
- **Stakeholders will speak a different language:**
  - Translation is required.



Identifying Architectural Characteristics from Domain Concerns

- **Example:** Translation from domain concerns to architectural concerns.

Domain concern	Architecture characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility



Identifying Architectural Characteristics from Requirements and Domain Knowledge

- **Example:** "A class registration system will be used by 1000 students to register for preferred class. The registration period is 10 hours."
  - *Might suggest the system must be able to handle 1000 users over a 10-hour period.*
- **Having previous experience with similar systems:**
  - You know many students will register in the final 10 minutes.
  - *Suggests that the system must be capable of dealing with bursts of use.*

Case Study: Identifying Architectural Characteristics

- **Let'work through the *Silicon Sandwich* example:**
- **Description:**
  - The Silicon Sandwich is a national sandwich shop that wants to enable online ordering (in addition to its current call-in service).
- **Users:** Thousands, perhaps one day millions.



Case Study – Requirements

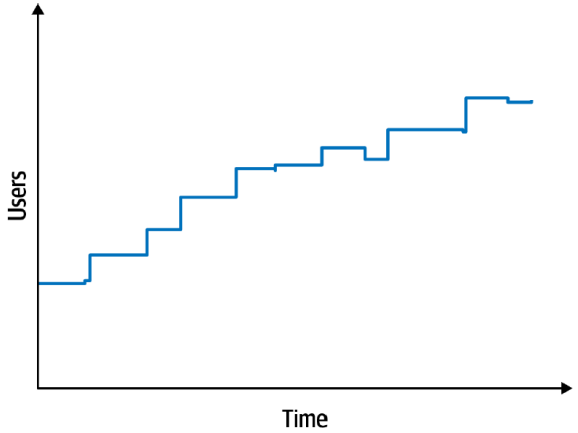
- **Requirements:**
  - Users will place their order, then be given a time to pick up their sandwich and directions to the store (which must integrate with several mapping services that include traffic information)
  - If the shop offers a delivery service, dispatch the driver with the sandwich to the user.
  - Mobile-device accessibility
  - Offer national daily promotions or specials.
  - Offer local daily promotions or specials.
- **Additional Context:**
  - Sandwich shops are franchised, each with a different owner.
  - Parent company has near-future plans to expand internationally.
  - Corporate goal is hiring low cost labor to maximize profit.

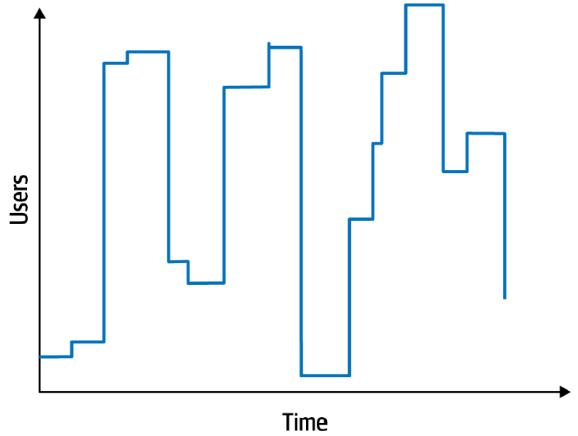
Case Study – Analysis

- **First thing to notice in the requirements is the user number:**

  "Thousands, perhaps one day millions."

  - This probably means that **scalability** is an important AC for the system, allowing the number of concurrent users without significant deterioration of performance.
  - Although not mentioned, **elasticity** (capability of handling bursts of data) can also be determined to be important, considering that we know the number of uses increases near meal time





Case Study – Analysis

- **Moving on to requirements:**
  - Some may suggest more than one AC, other may suggest none.

    Users will place their order, then be given a time to pick up their sandwich and directions to the store (which must integrate with several mapping services that include traffic information).

  - **External mapping services imply integration points, which may impact reliability.**
    - If the external service fails…
      - Will the Silicon Sandwich service also fail?
      - Will it operate with diminished capacity (e.g. without traffic information)



Case Study – Analysis

Requirements (Continued)

If the shop offers a delivery service, dispatch the driver with the sandwich to the user.

- **This does not appear to need support from specific architectural characteristics**

Mobile-device accessibility

- **This will mostly impact the design of the application.**
  - It can be created as a web application or several native apps:
  - Creating and maintaining multiple native apps will probably be overkill.
  - May lead to specification of targets such as page-loading performance.
  - Working with stakeholders important to check any such decisions.



Case Study – Analysis

Requirements (Continued):

> Offer national daily promotions/specials.
> Offer local daily promotions/specials

- Both requirements specify **customizability** across both promotions and specials (address specific traffic advice also suggests the same).
  - *This can be directly supported by the architecture using a plug-in system on a microkernel architecture*
  - *But a design-oriented solution using traditional OO design patterns such as the Template Method can also be considered.*



Case Study – Analysis

Requirements (Continued):

Accept payment online, in person, or upon delivery.

- Suggests **security** is important, but nothing suggests an exceptional care beyond what is usually implied by default.

Sandwich shops are franchised, each with a different owner..

- This suggests cost is an important factor. We should consider if a simple/throwaway architecture is enough.

Case Study – Analysis

- **Requirements (Continued):**

  Parent company has near-future plans to expand overseas.

  - Implies **internationalization** is necessary. It will impact low-level design, but structural impact should be minimal.

  Corporate goal is to hire inexpensive labor to maximize profit.

  - **Usability** may be important, but that is it is ultimately mostly driven by design.

- While **performance** is not explicitly mentioned, we know it must be maintained, especially during peak hours.



Case Study – Analysis

- **Implicit characteristics that can be considered:**
  - **Availability –** making sure users can access the site.
  - **Reliability –** Ensuring site stays up during interactions.
  - **Security –** No one wants to create insecure software…
  - **Customizability** – Lots of things can change: recipes,promotions, directions..
- **Choices must be made:**
  - Under-specification is bad…
  - Over-specification is bad



Case Study – Analysis

- **So, what should be prioritized here?**
- **You can relax, because there are no right answers…**
  - All answers are incorrect, or to put in in another way…

There are no wrong answers in architecture, only expensive ones.



Case Study – Design vs. Architecture

- In the specific case of **customizability,** we can find alternative solutions:
  - **Architecture-oriented:** Microkernel with plugins
  - **Design-oriented:** Template method pattern
- **How to choose? Consider the trade-offs:**
  - *Are the disadvantages of Microkernal architecture (performance, coupling) relevant for other aspects of the system?*
  - *Are other desirable characteristics more difficult in one solution than the other?*
  - *What is the cost to support either solution?*



Case Study – Towards a simpler architecture

- **Always try to simplify as much as possible.**
- **After finding a solution, try to review it:**
  - What is the *least* important characteristic?
    - In this case, perhaps we could select **customizability** or **performance**.
    - If we dropped **customizability**, then it can be implemented as in the design-space.
    - If we dropped **performance**, that does not mean we want to build a slow system, but that one does not chose to sacrifice other important characteristics such as scalability or availability for achieving higher performance.

1.3

Measuring and Governing Architectural Characteristics

How to measure and control the state of important ACs.



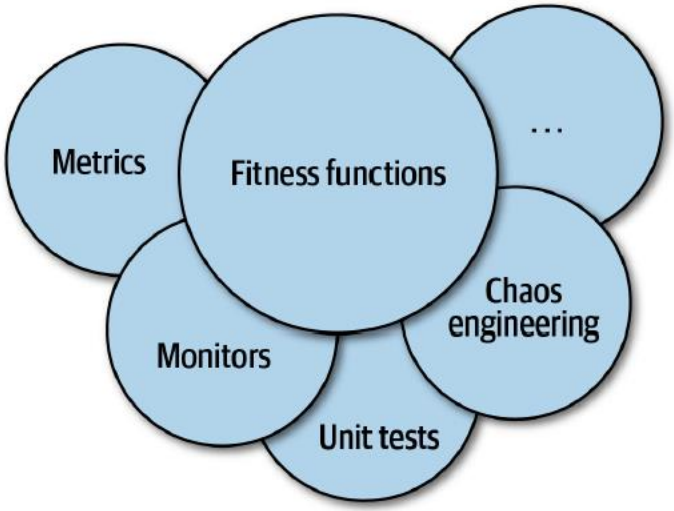
Measuring Architecture Characteristics

- **Operational Measures**
  - **Most can be measured directly:**
    - Performance, Scalability
  - **What is it necessary to measure?**
    - Are we interested in *average* response time?
    - What if 1% of requests take 10x the average? Is that ok?
- **Structural Measures**
    - Compute code quality metrics
    - Inspect code structure.



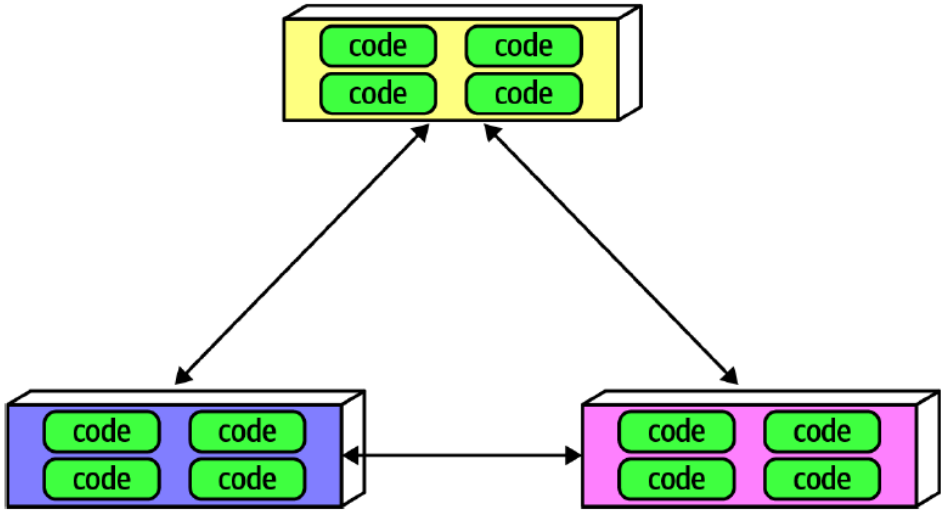
Measuring Architecture Characteristics

- A fitness function is sometimes used that aggregates multiple measurements/analysis from different sources into a single quality metric.



Example – Cyclic Dependencies

- A cyclic dependency is an anti-pattern that dramatically reduces code modularity.





Example – Cyclic Dependencies

- A fitness function can be implemented as an Unit test that checks structural properties.
  - Example uses the *JDepend* tool for dependency analysis.

```
public class CycleTest {
    private JDepend jdepend;

    @BeforeEach
    void init() {
          jdepend = new JDepend();
          jdepend.addDirectory("/path/to/project/persistence/classes");
          jdepend.addDirectory("/path/to/project/web/classes");
          jdepend.addDirectory("/path/to/project/thirdpartyjars");
    }

    @Test
    void testAllPackages() {
          Collection packages = jdepend.analyze();
          assertEquals("Cycles exist", false, jdepend.containsCycles());
          }
      }
```



Example – Distance from Main Sequence

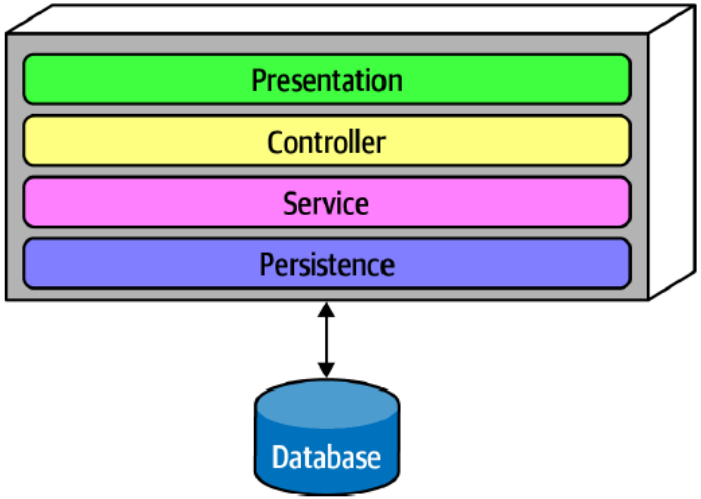
- A fitness function can be implemented as Unit test that checks structural properties.
  - Example uses the *JDepend* tool for dependency analysis.

```
@Test
void AllPackages() {
    double ideal = 0.0;
    double tolerance = 0.5; // project-dependent
    Collection packages = jdepend.analyze();
    Iterator iter = packages.iterator();
    while (iter.hasNext()) {
      JavaPackage p = (JavaPackage)iter.next();
      assertEquals("Distance exceeded: " + p.getName(),
            ideal, p.distance(), tolerance);
    }
}
```



Example – Checking Architectural Constraints

- In a layered architecture, layers can never depend on higher-level layers.





Example – Checking Architectural Constraints

- A test can be created, with the ArchUnit tool, to enforce the Architectural constraints.

```
layeredArchitecture()
    .layer("Controller").definedBy("..controller..")
    .layer("Service").definedBy("..service..")
    .layer("Persistence").definedBy("..persistence..")

    .whereLayer("Controller").mayNotBeAccessedByAnyLayer()
    .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
    .whereLayer("Persistence").mayOnlyBeAccessedByLayers("Service")
```

Fitness function verifying layered architecture constraints

1.3

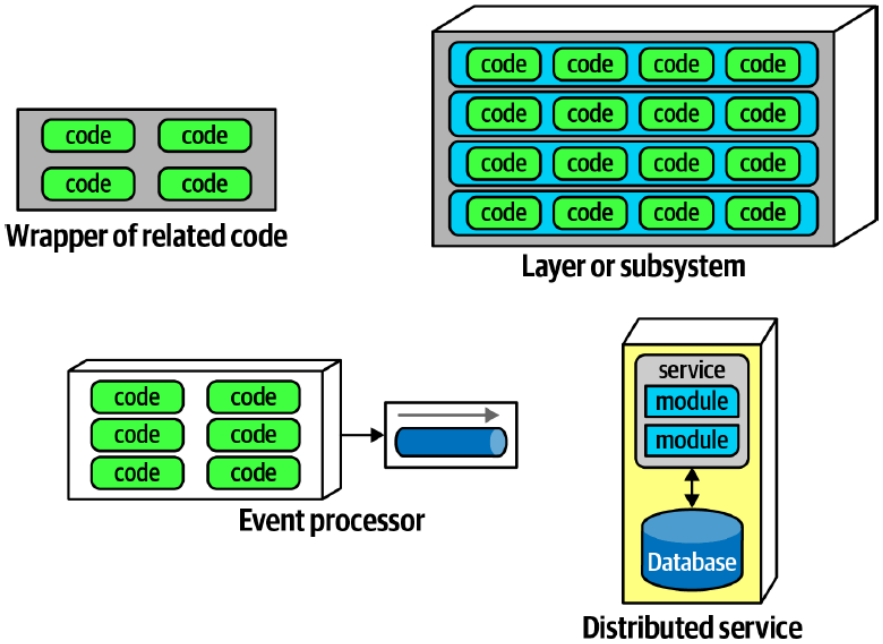
Component-Based Thinking

Thinking in terms of components

Components

Different types of components

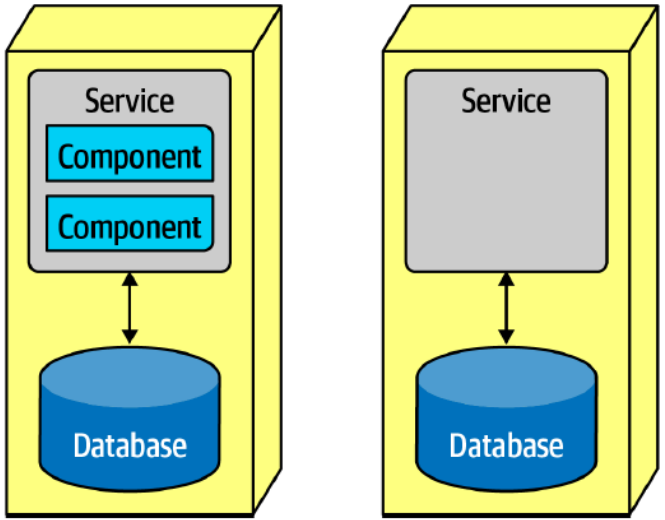
- Components form the fundamental modular building block in architecture





Components

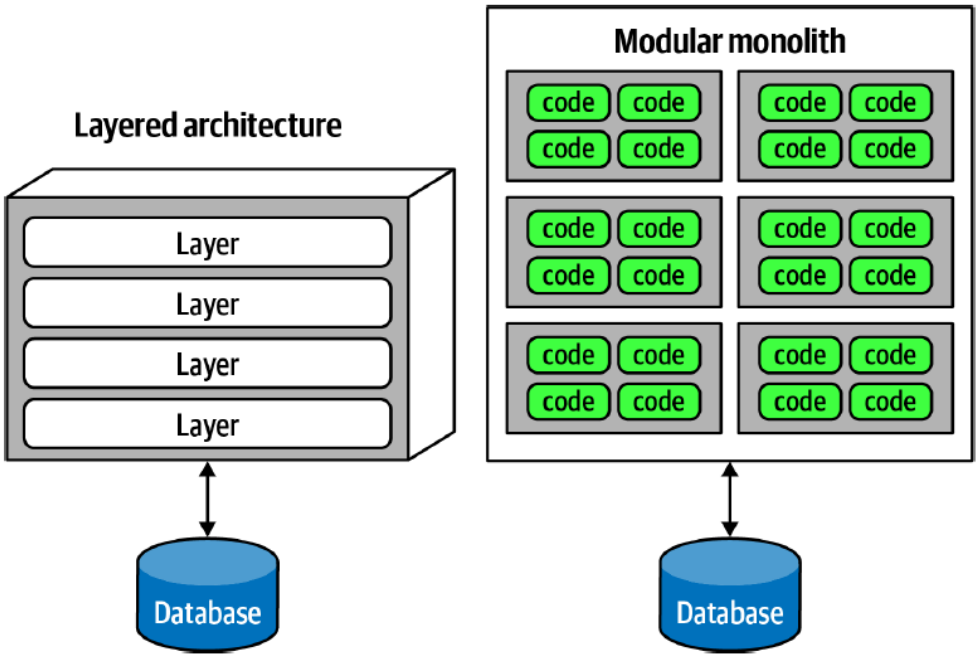
- Components are defined by convenience at a high level of abstraction.
  - A service may include subcomponents..
  - Or no subcomponents at all if it is simple enough





Architecture Partitioning

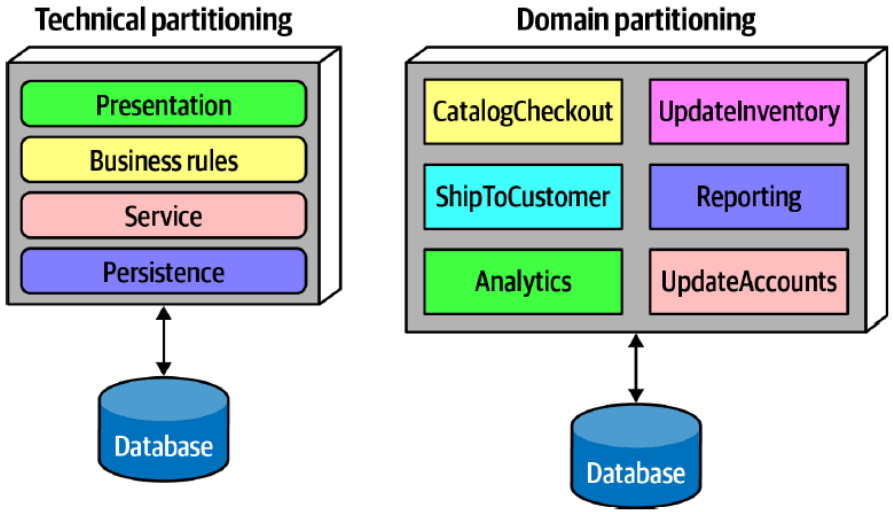
- Components are used to define the *top-level partitioning* in an architecture.
  - These are usually based on well known *architectural styles.*





Architecture Partitioning

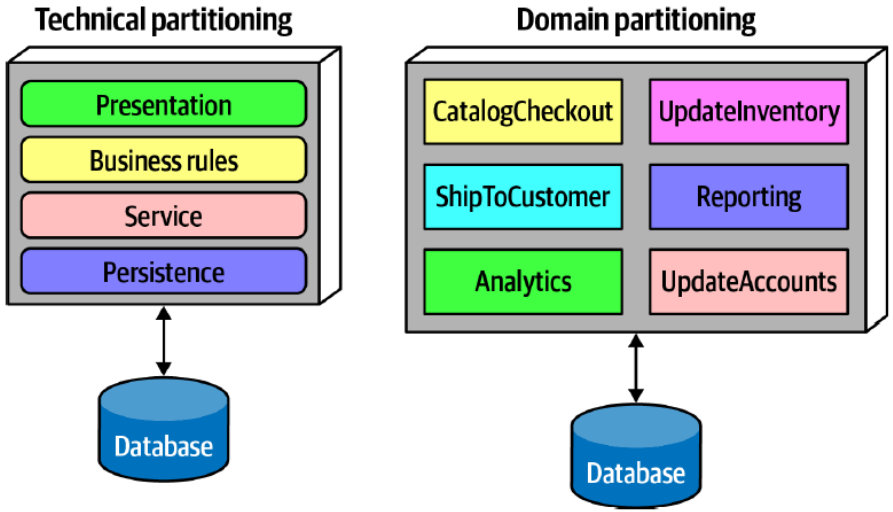
- Different approaches can be adopted when identifying components in the top level partition:





Architecture Partitioning

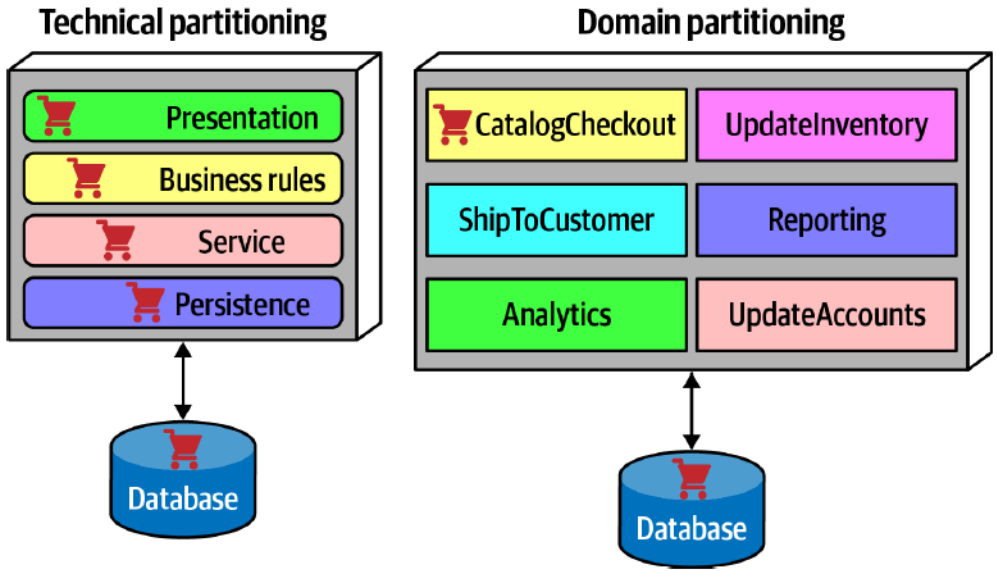
- Different approaches can be adopted when identifying components in the top level partition:





Architecture Partitioning

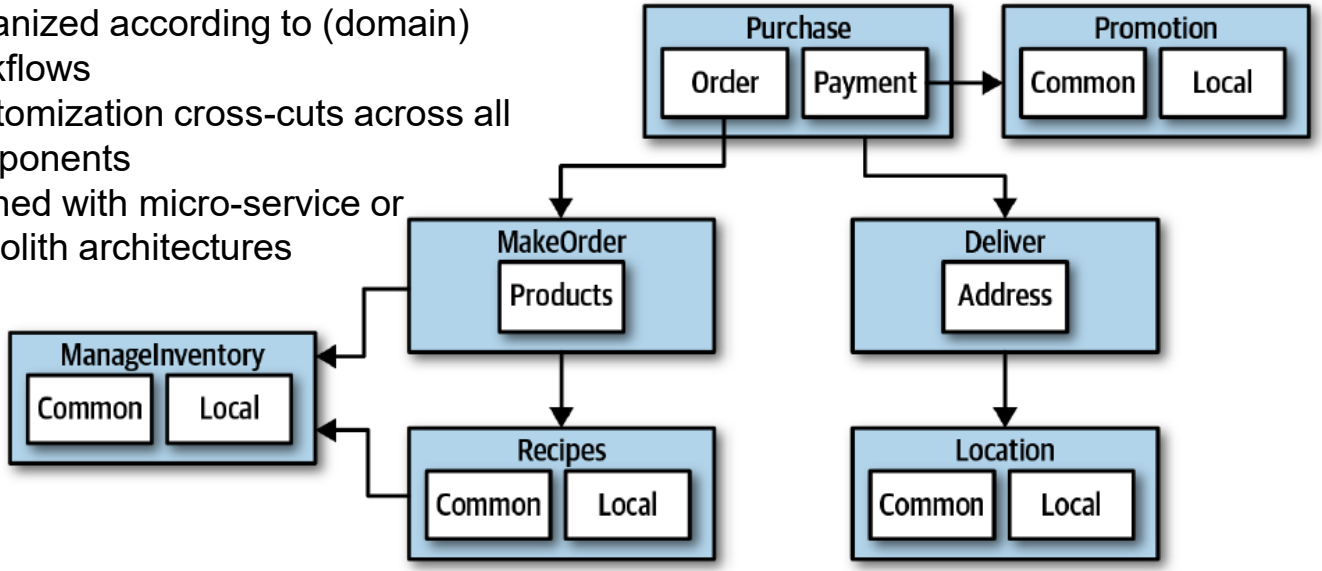
- Domains and workflows are represented differently in each approach:



Case Study: Silicon Sandwiches: Partitioning

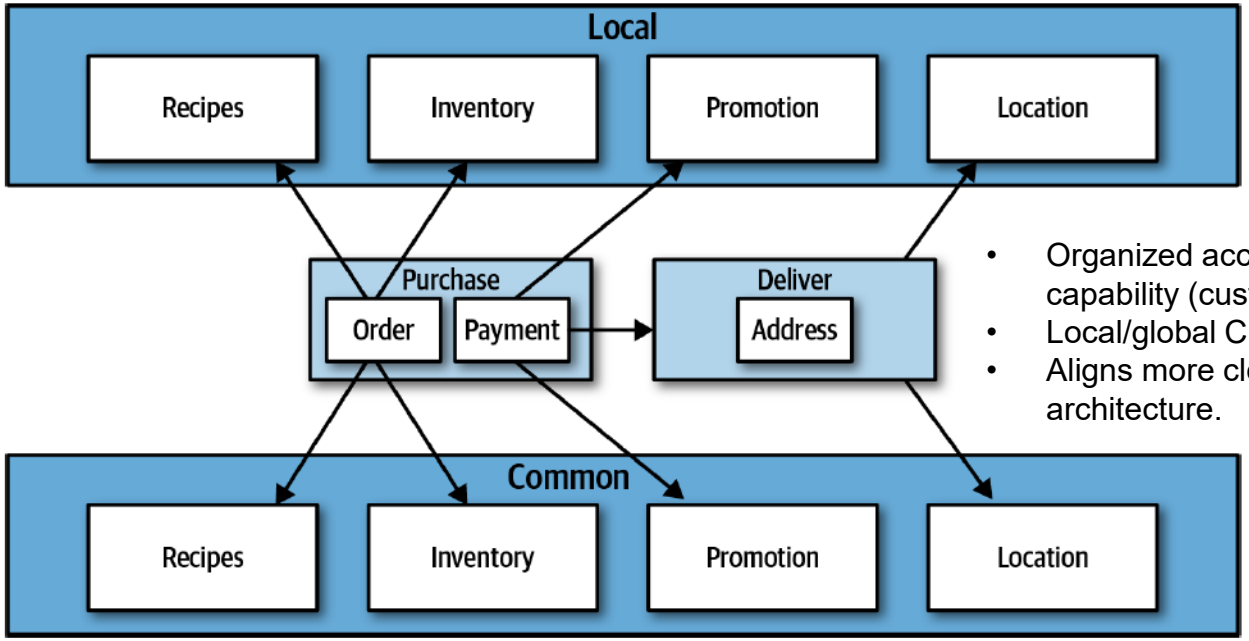
Domain Partitioned Design

- Organized according to (domain) workflows
- Customization cross-cuts across all components
- Aligned with micro-service or monolith architectures



Case Study: Silicon Sandwiches: Partitioning

Alternative Design



- Organized according to technical capability (customization)
- Local/global Customization separated
- Aligns more closely with layered architecture.



Domain Partitioning

- **Advantages:**
  - Modelled more closely toward how the business functions rather than an implementation detail
  - Easier to utilize the *Inverse Conway Maneuver* to build cross-functional teams around domains
  - Aligns more closely to the modular monolith and microservices architecture styles
  - Message flow matches the problem domain
  - Easy to migrate data and components to distributed architecture
- **Disadvantages:**
  - **Customization code appears in several places**

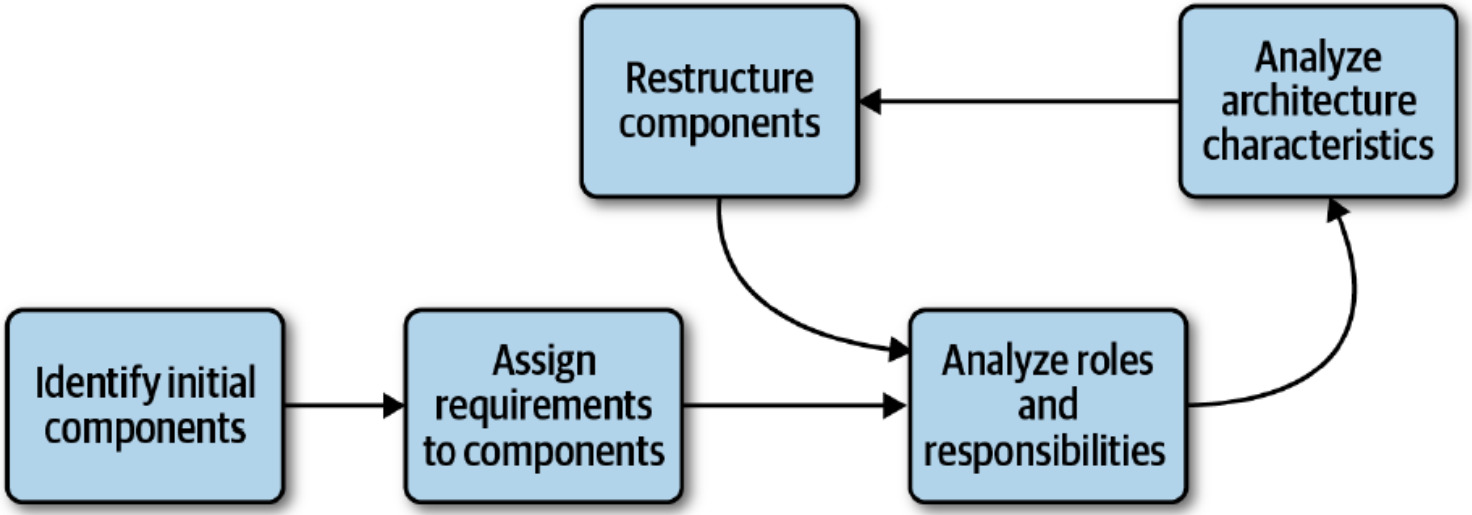


Technical Partitioning

- **Advantages:**
  - Clearly separates customization code.
  - Aligns more closely to the layered architecture pattern.
- **Disadvantages:**
  - Higher degree of global coupling. Changes to either the Common or Local component will likely affect all the other components.
  - Developers may have to duplicate domain concepts in both common and local layers.
  - Typically higher coupling at the data level. In a system like this, the application and data architects would likely collaborate to create a single database, including customization and domains. That in turn creates difficulties in untangling the data relationships if the architects later want to migrate this architecture to a distributed system.

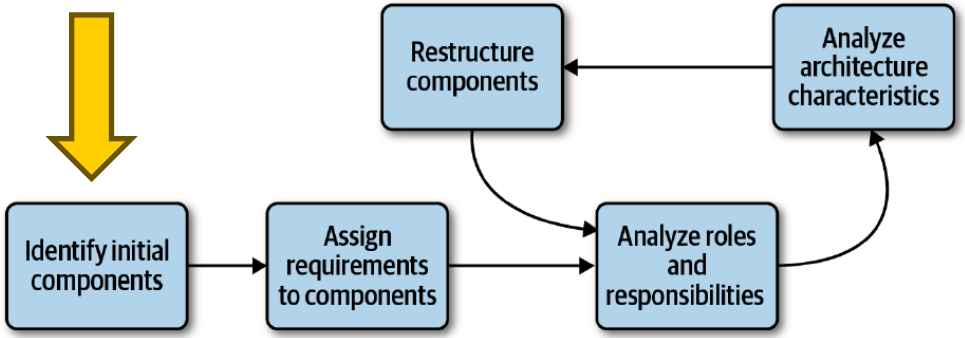


Component Identification Flow



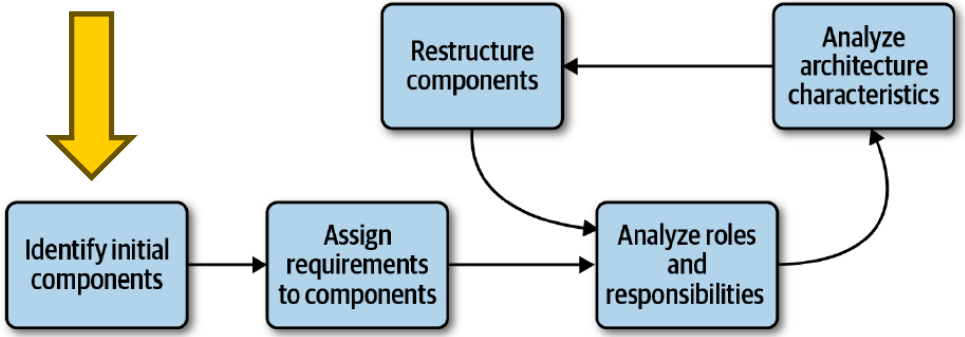
Component Identification Flow.

- First, freely arbitrate some top-level components based on the chosen top-level partition
  - This will probably not lead directly to a good solution, but that's ok.
  - It's just a starting point.
  - This design will be iterated on to produce a good solution



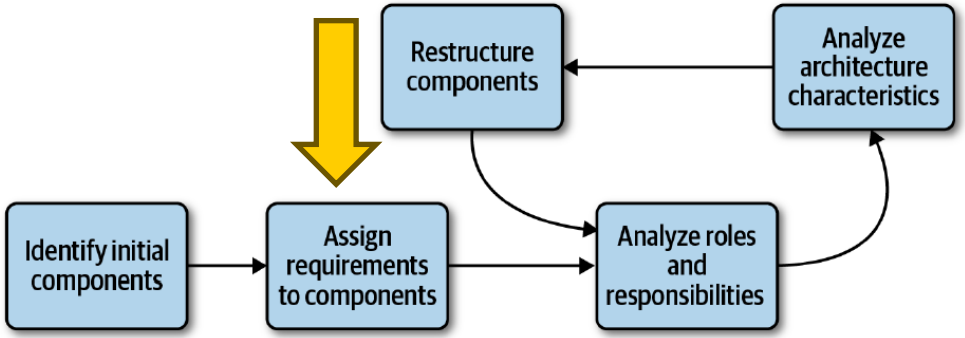
Component Identification Flow.

- First, freely arbitrate some top-level components based on the chosen top-level partition
  - This will probably not lead directly to a good solution, but that's ok.
  - It's just a starting point.
  - This design will be iterated on to produce a good solution



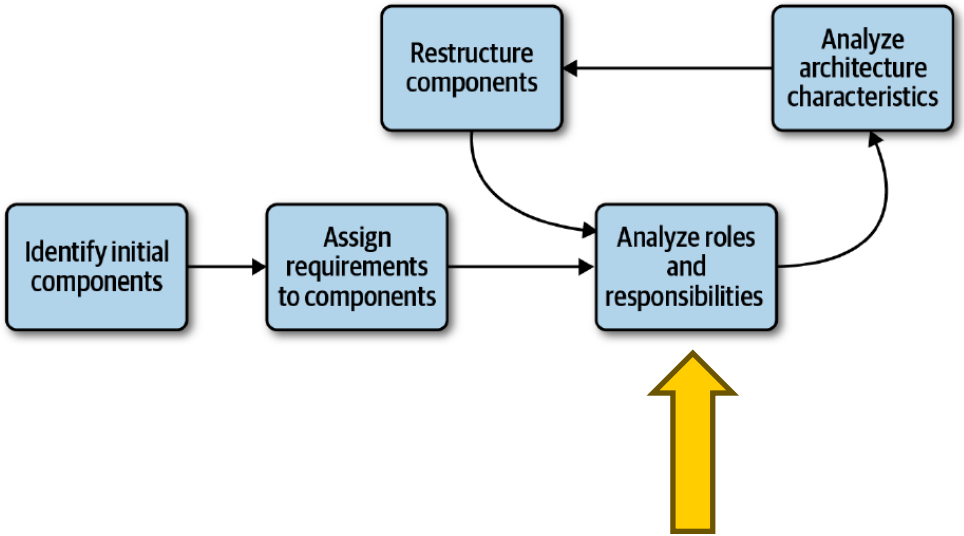
Component Identification Flow.

- Then, align requirements to components.
  - How well do they fit?
  - May lead to merging, creating or splitting components so that a sensible mapping is found.



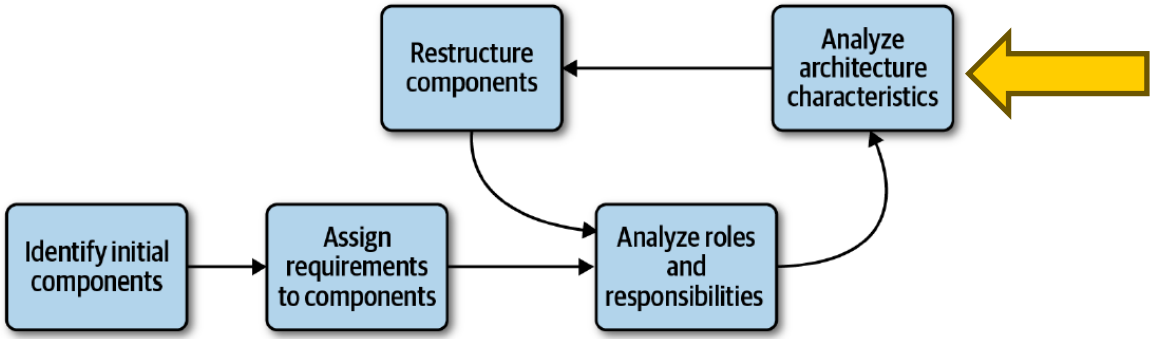
Component Identification Flow.

- Consider roles and responsibilities identified in requirements
  - Does the granularity fit?
  - Component granularity should approximately match domain granularity.



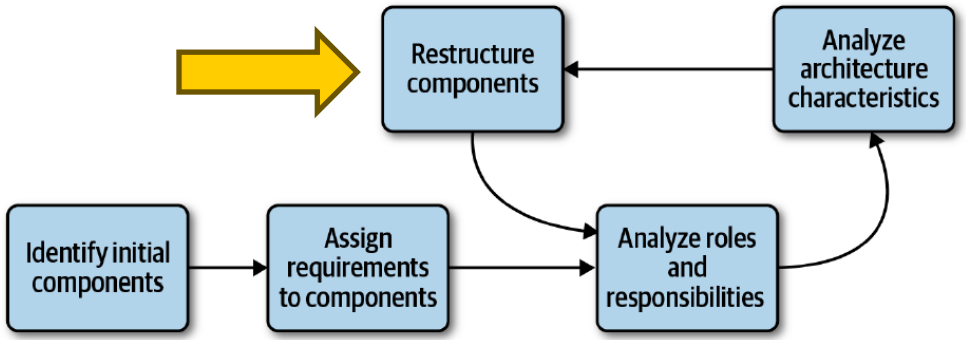
Component Identification Flow.

- Consider architectural characteristics:
  - Consider if the previously identified architectural characteristics somehow impact component granularity or division.
  - For example, if a part of the system deals with thousands of concurrent users and other part deals with much few, then even if functionally these parts are similar, they might require separated structure due to differences in the architectural characteristics.



Component Identification Flow.

- **Restructure the solution:**
  - As development progresses, the initial solution is continually refined with the feedback from developers, ensuring architectural analysis remains relevant and consistent.



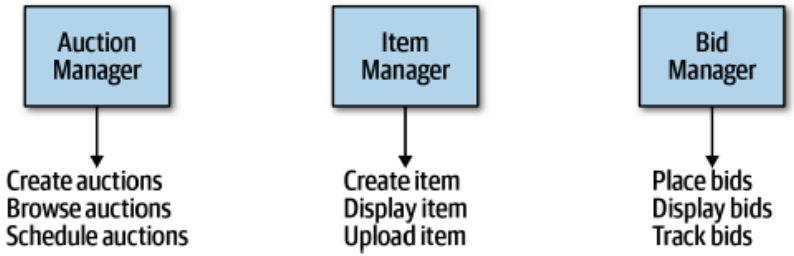


Component Granularity

- **We want to identify an ideal level of component granularity**
  - **Too fine-grained** – Too much communication and detail for same results
  - **Too coarse-grained** – High internal coupling, makes testing and deployment difficult
- **Easier said than done.**
- **The goal is to partition the problem space in coarse chunks that correspond to architectural characteristics.**

Component Granularity

- Entity “Trap”:
  - Creating a Manager object for each entity.
  - Each component responsible for its CRUD operations.
  - This conflates the workflows with DB accesses.
- This is essentially an Object-Relational mapping (ORM) of a framework to a database.
- If all that is required is a simple mapping of user interface to DB, then a suitable framework can be used instead. E.g.
  - NakedObjects for C#
  - Isis for Java
  - Scaffolding on Ruby on Rails





Component Granularity

- **Actor/Actions Approach**
  - Identity actors who perform actions on the system.
  - Identify the activities actors perform.
- **Event Storming**
  - Identify workflows and assume the project uses messages and/or events.
  - Then, identify events based on requirements and identified roles.
  - Components are created around event/message handlers
- Wor**kflow Approach**
  - Model components based on identified workflows.
  - Does not assume messaging or events.
  - Build components based on identified roles and activities.

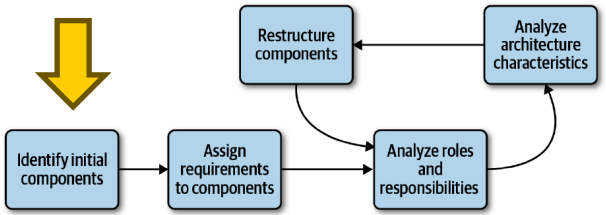


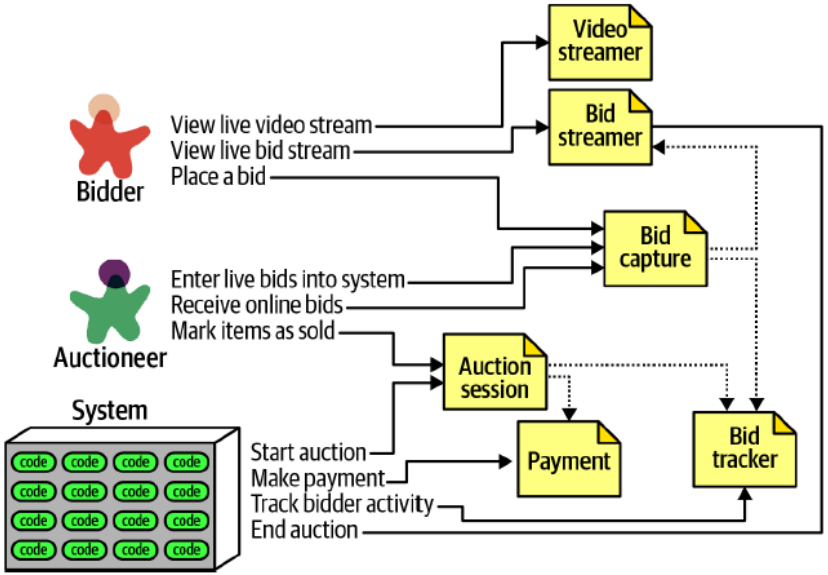
Case Study: Auction system

- **We have no specific constraints, lets try to identify a good general purpose architecture using the Actor/actions approach**
- **The system has three obvious roles:**
  - **Bidder** – Makes bids
  - **Auctioneer** – Puts items up for auction
  - **System** – Responsible for triggering internal actions
- Possible set of actions:
  - **Bidder:** View live video stream, view live bid stream, place a bid
  - **Auctioneer:** Enter live bids into system, receive online bids, mark item as sold
  - **System:** Start auction, make payment, track bidder activity

Case Study: Auction system

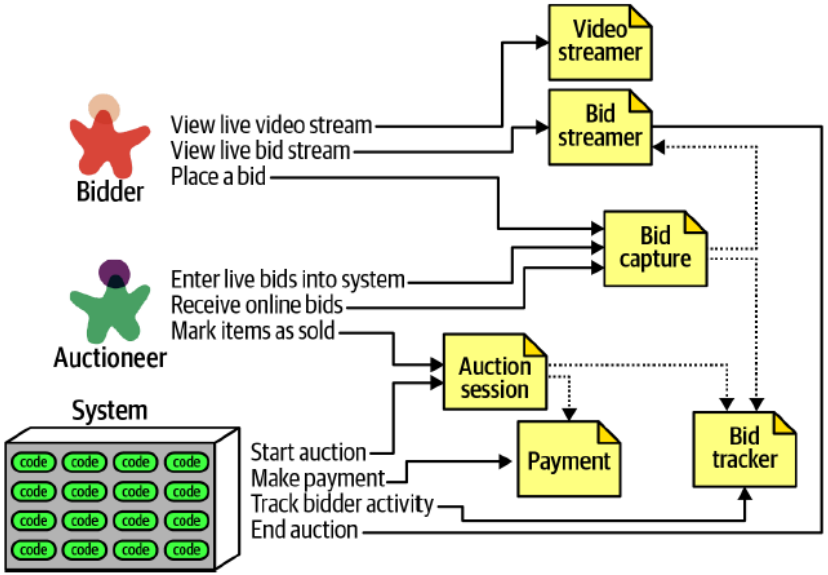
- We can begin by using the identified information to find a starter set of components.
- Each role and action maps to a component, which then collaborate, as necessary.





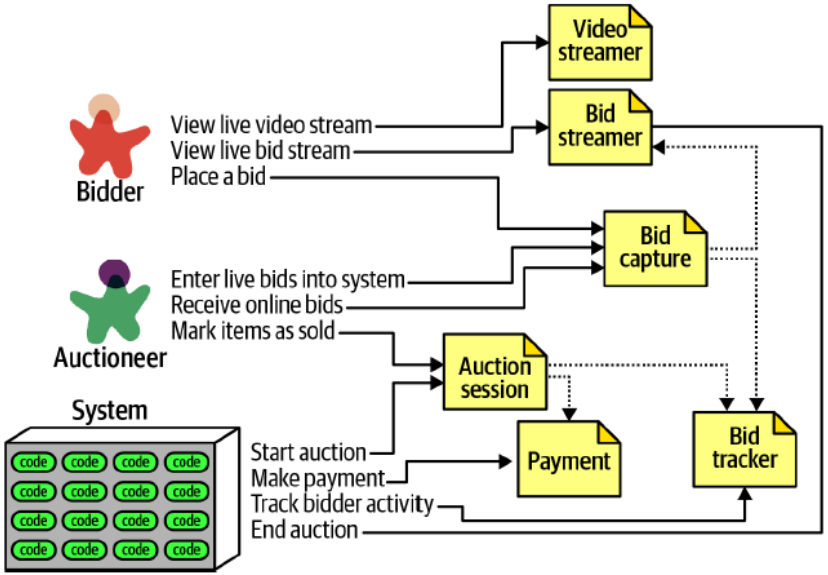
Case Study: Auction system

- **Videostreamer:** Streams a live auction to users.
- **BidStreamer:** Stream bids as they occur to the users. Both VideoStreamer and BidStream offer read-only views to users.
- **BidCapture:** Captures bids from both auctioneer and users.
- **BidTracker:** Tracks and records bids
- **Auction Session:** Starts and stops and auction. When bidder ends the auction, performs payment and resolution, including notifications.
- **Payment:** Third-party payment processor.



Case Study: Auction system

- Consider relevant architectural characterist and see how they map to components.
- For example, **BidCapture** is responsible for capturing bids from both auctioneer and us
  - This makes sense *functionally.*
- But what are the applicable ACs?
  - The auctioneer does ***not*** require the same scalability or elasticity level as potentially thousands of users.
  - Reliability and availability can be ***more*** important for auctioneer than for a normal bidder. It's not good if bidders can't login or it they suffer a dropped connection, but it is disastrous if the same happens to auctioneer.



Case Study: Auction system

- This suggests that the Bid Capture module needs to account for different sets of relevant ACs for different users.
- This leads to splitting the Bid Capture into two different components.
  - Bid Capture
  - Auctioneer Capture
- A similar analysis can be applied to other parts of the system.
- This is probably a good starting point, but further iteration might be necessary.

